

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

**«Национальный исследовательский университет
ИТМО»**

Факультет программной инженерии и компьютерной техники

Дисциплина

«Функциональная схемотехника»

Лабораторная работа №3

«Расширение возможностей учебного процессорного ядра schoolRISCV»

Вариант «FIFO»

Выполнили:

Андреева Б.В., группа Р3331

Колмаков Д.В., группа Р3331

Преподаватель:

Васильев С.Е.

Санкт-Петербург

2024 г.

Содержание

Содержание.....	2
Задание.....	3
Ход работы.....	4
Микроархитектурная схема процессорного ядра.....	4
Описание микроархитектурной схемы и реализованных изменений.....	4
Описание алгоритма функционирования микропроцессорного ядра в части исполнения ваших инструкций.....	5
Результаты отчёта по временным параметрам.....	6
Выводы.....	7

Задание

1. Модифицировать процессорное ядро для реализации очереди по принципу FIFO, реализовать две команды:
 - a. **push xN** - должна загружать данные из младшей части регистра xN в очередь;
 - b. **pop xN** - должна выгружать данные в младшую часть регистра xN.
2. Подготовить тестовое окружение системного уровня и убедиться в корректности реализации путём запуска симуляционных тестов.

Микроархитектурная схема процессорного ядра



Входы:

- **clk** - тактовый сигнал clock;
- **reset** - сигнал сброса;
- **writeEnable** - вход разрешения записи в очередь, подключен к линии *fifoPush* из управляющего устройства;
- **writeData** - 32х-битная шина входных данных, подключена по линии *fifoIn* к выходу *RD1* блока регистров;
- **readEnable** - вход разрешения чтения из очереди, подключен к линии *fifoPop* из управляющего устройства.

- **readData** - 32х-битная шина выходных данных, подключена по линии *fifoOut* ко входу *WD3* блока регистров.

Описание алгоритма функционирования микропроцессорного ядра в части исполнения ваших инструкций

Были реализованы две команды: запись и чтение из очереди.

Запись в очередь реализована в виде команды **sw rs1, imm(rs2)**. При этом запись доступна из регистра, номер которого указан в поле *rs2*, а поле *rs1* не используется, так как запись происходит в очередь, а не в другой регистр. Поле *imm* должно быть равно 0. Таким образом, команда имеет вид:

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
0000000	XXXXX	?????	010	00000	0100011

При поступлении команды **sw** блок управления подает сигнал *fifoPush* = 1 на вход очереди *writeEnable*. Данные для записи подаются из выхода *RDI* блока регистров на вход *writeData*. По тактовому сигналу происходит запись данных в очередь.

Чтение из очереди реализовано в виде команды **lw rd, imm(rs1)**. При этом запись доступна только в регистр, номер которого указан в поле *rd*, а поле *rs1* не используется, так как чтение происходит из очереди, а не регистра. Таким образом, команда имеет вид:

imm[11:0]	rs1	funct3	rd	opcode
0000000000000	?????	010	XXXXX	0000011

При поступлении команды **lw** блок управления подает сигнал *fifoPop* = 1 на вход очереди *readEnable*. Данные из очереди подаются на мультиплексор перед входом *WD3* блока регистров. Также блок управления устанавливает значение *wdSrc* = 10₂, тем самым мультиплексор подает данные из очереди на вход блока регистров. Значение сигнала *regWrite* = 1. По тактовому сигналу происходит чтение данных из очереди.

Результаты отчёта по временным параметрам

Setup время — это минимальное время, которое сигнал должен удерживаться на входе регистра до такта перехода для корректного захвата. Так как значение WNS положительное, это значит, что времени достаточно для корректной установки сигнала.

Hold время — это минимальное время, которое сигнал должен оставаться на входе регистра после такта, чтобы правильно установиться. Так как WHS положителен, это указывает на успешное выполнение временных ограничений.

Pulse Width время — минимальная длительность высокого или низкого уровня сигнала. Так как WPWS положителен, это значит, что времени достаточно для корректной установки сигнала

TNS — общий суммарный недостаток установки. **THS** — общий запас удержания. **TPWS** — общий недостаток ширины импульса. Так как значения равны нулю, это означает отсутствие проблем.

Number of Failing Endpoints — количество точек, которые не соответствуют временным ограничениям. В данном случае их 0, что указывает на отсутствие ошибок.

Total Number of Endpoints — общее количество конечных точек в проекте (187 и 143 для разных групп сигналов).

Выводы

(а) Описать достоинства и недостатки ваших модификаций;

Среди достоинств наших модификаций можно выделить следование архитектуре RISC-V, простота реализации, корректность. Реализация была проверена с помощью симуляционных тестов, что подтверждает её корректность и надежность. Из недостатков - отсутствие аутентичных команд `push` и `pop`, которые повысили бы простоту написания и читаемость кода, так как эти команды не предназначены для работы с очередями.

(b) Привести альтернативные решения для ваших модификаций (для дополнительного задания необязательно);

- Реализация собственных команд `push` и `pop`, что упрощает написания ассемблерного кода;
- Реализация очереди на уровне программного обеспечения, что позволяет избежать изменений в микроархитектуре.

(c) Дать ответ на вопрос, чем ваше решение лучше альтернатив и почему для реализации было выбрано именно оно;

- Реализация очереди FIFO на аппаратном уровне обеспечивает высокую производительность и минимальные задержки при обработке данных;
- А использование существующих команд `sw` и `lw` упрощает интеграцию очереди в существующую микроархитектуру и минимизирует необходимые изменения.

(d) Перечислить все возникшие в ходе выполнения работы вопросы.

- Почему vivado так долго синтезирует дизайн?
- Почему vivado так долго имплементирует дизайн?